

Python Programming and Introduction to Algorithms

Basics & Algorithms

Hoa T. Vu

Variables

Variables

A variable stores a value. Python infers the type automatically.

```
x = 10          # int
pi = 3.14       # float
name = "Alice"  # str
flag = True     # bool
```

```
print(x)        # 10
print(type(x))  # <class 'int'>

x = x + 1       # reassign
x += 1          # shorthand
```

Variables

A variable stores a value. Python infers the type automatically.

```
x = 10          # int
pi = 3.14       # float
name = "Alice"  # str
flag = True     # bool

print(x)        # 10
print(type(x))  # <class 'int'>

x = x + 1       # reassign
x += 1          # shorthand
```

- No declaration needed, just assign
- Types can change (dynamic type): `x = 10` then `x = "hello"` is valid

Common Operations

Arithmetic

```
7 + 2    # 9
7 - 2    # 5
7 * 2    # 14
7 / 2    # 3.5    (float)
7 // 2   # 3      (floor div)
7 % 2    # 1      (remainder)
7 ** 2   # 49     (power)
```

Strings

```
s = "hello"
len(s)                # 5
s.upper()             # "HELLO"
s[0]                  # "h"
s[1:4]                # "ell"

# format string
n = 42
f"answer: {n}"        # "answer: 42"
```

Lists

Lists

An ordered, mutable collection of items.

```
nums = [3, 1, 4, 1, 5]
```

```
nums[0]          # 3          (first element)
```

```
nums[-1]         # 5          (last element)
```

```
nums[1:3]        # [1, 4]    (slice)
```

```
len(nums)        # 5
```

```
nums.append(9)    # [3, 1, 4, 1, 5, 9]
```

```
nums.pop()        # removes and returns 9
```

```
nums.sort()       # sorts in place: [1, 1, 3, 4, 5]
```

Lists

An ordered, mutable collection of items.

```
nums = [3, 1, 4, 1, 5]
```

```
nums[0]          # 3          (first element)
```

```
nums[-1]         # 5          (last element)
```

```
nums[1:3]        # [1, 4]    (slice)
```

```
len(nums)        # 5
```

```
nums.append(9)    # [3, 1, 4, 1, 5, 9]
```

```
nums.pop()        # removes and returns 9
```

```
nums.sort()       # sorts in place: [1, 1, 3, 4, 5]
```

Lists can hold mixed types: [1, "hello", 3.14, True]

List Comprehension

Build a list in one line more readable than a loop.

```
# squares of 0..4  
squares = [x**2 for x in range(5)]  
# [0, 1, 4, 9, 16]
```

```
# even numbers only  
evens = [x for x in range(10) if x % 2 == 0]  
# [0, 2, 4, 6, 8]
```

```
# nested: 2-D grid of (row, col) pairs  
pairs = [(r, c) for r in range(3) for c in range(3)]
```

Dictionaries

Dictionaries

A mapping from **keys** to **values**.

```
student = {  
    "name": "Alice",  
    "age": 22,  
    "gpa": 3.8,  
}
```

```
student["name"]          # "Alice"  
student["age"] = 23      # update  
student["major"] = "CS"  # add new key
```

```
"gpa" in student         # True  
student.get("phone", "N/A") # "N/A" if missing
```

Dictionaries

A mapping from **keys** to **values**.

```
student = {  
    "name": "Alice",  
    "age": 22,  
    "gpa": 3.8,  
}
```

```
student["name"]          # "Alice"  
student["age"] = 23      # update  
student["major"] = "CS"  # add new key
```

```
"gpa" in student          # True  
student.get("phone", "N/A") # "N/A" if missing
```

Keys must be *immutable* (strings, numbers, tuples). Values can be anything.

Iterating Dictionaries

```
grades = {"Alice": 90, "Bob": 85, "Carol": 92}
```

```
# keys
```

```
for name in grades:  
    print(name)
```

```
# values
```

```
for score in grades.values():  
    print(score)
```

```
# both
```

```
for name, score in grades.items():  
    print(f"{name}: {score}")
```

Conditionals

if / elif / else

Python uses **indentation** (4 spaces) to define blocks : no braces.

```
score = 78

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"

print(grade)  # "C"
```

Comparison Operators

- `==` equal `!=` not equal
- `<` `<=` `>` `>=` ordering
- `and` `or` `not` logical
- `in` membership test

```
x = 5
x > 3 and x < 10    # True
x in [1, 3, 5, 7]   # True
not (x == 4)        # True
```


Loops

for Loop

iterate over a list

```
for x in [2, 4, 6]:  
    print(x)
```

iterate with index

```
for i, x in enumerate([2, 4, 6]):  
    print(i, x)
```

range(start, stop, step)

```
for i in range(5):          # 0, 1, 2, 3, 4  
    print(i)
```

```
for i in range(0, 10, 2):   # 0, 2, 4, 6, 8  
    print(i)
```

while Loop

```
n = 1
while n < 100:
    n *= 2
print(n)    # 128
```

break and continue

```
for x in range(10):
    if x == 5:
        break           # exit the loop entirely
    if x % 2 == 0:
        continue        # skip to next iteration
    print(x)             # prints 1 3
```

- Python is widely used for data science and machine learning development.

- Python is widely used for data science and machine learning development.
- Python can be slow, but a lot of underlying libraries were written in fast languages (e.g., C++)

- Python is widely used for data science and machine learning development.
- Python can be slow, but a lot of underlying libraries were written in fast languages (e.g., C++)
- You can make Python code faster using Cython.

Introduction to Algorithms

- Sequence of instructions that solve a problem
- Running time: number of computer instructions (memory access and pairwise arithmetics)

Selection Sort

- Find the **minimum** element in the unsorted portion
- **Swap** it into the first unsorted position
- Repeat, shrinking the unsorted portion by one each time



Selection Sort : Code

```
def selection_sort(a):  
    n = len(a)  
    for i in range(n):  
        next_min = i  
        for j in range(i + 1, n):  
            if a[j] < a[next_min]:  
                next_min = j  
        a[i], a[next_min] = a[next_min], a[i]  
  
a = [5, 3, 8, 1, 4]  
selection_sort(a)  
# [1, 3, 4, 5, 8]
```

Selection Sort : Code

```
def selection_sort(a):  
    n = len(a)                                #1  
    for i in range(n):                        #2  
        next_min = i                          #3  
        for j in range(i + 1, n):            #4  
            if a[j] < a[next_min]:             #5  
                next_min = j                  #6  
        a[i], a[next_min] = a[next_min], a[i] #7
```

Selection Sort : Running Time

We count the number of times each line runs:

- Lines #1, #3, #7: run n times (once per outer iteration)
- Lines #4–#6: inner loop runs

$$(n-1) + (n-2) + \cdots + 1 + 0 = \frac{n(n-1)}{2}$$

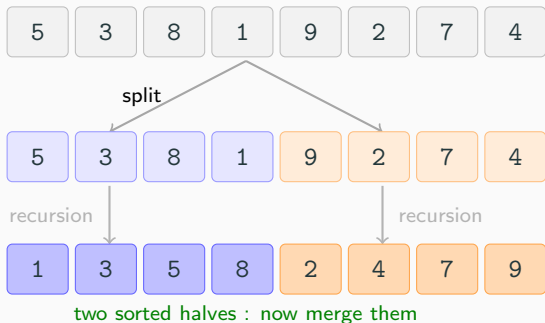
times in total

- All other lines: constant or $O(n)$: negligible

Total $\approx \frac{n^2}{2}$ operations $\Rightarrow$ **running time** = $O(n^2)$.

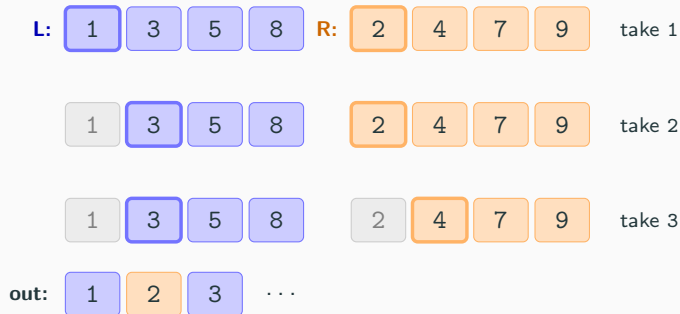
Merge Sort : Recursion

- **Split** the array into two halves
- **Recursively** sort each half (trust the recursion!)
- **Merge** the two sorted halves (next slide)



Merge Sort : Merge Step

Compare front elements of L and R; take the smaller one. Repeat.



Merge Sort : Code (merge)

```
def merge(left, right):
    out = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            out.append(left[i])
            i += 1
        else:
            out.append(right[j])
            j += 1
    out += left[i:]    # append remaining elements
    out += right[j:]
    return out
```

Merge Sort : Code (merge sort)

```
def merge_sort(a):  
    if len(a) <= 1:          # base case  
        return a  
    mid = len(a) // 2  
    left  = merge_sort(a[:mid])  
    right = merge_sort(a[mid:])  
    return merge(left, right)
```

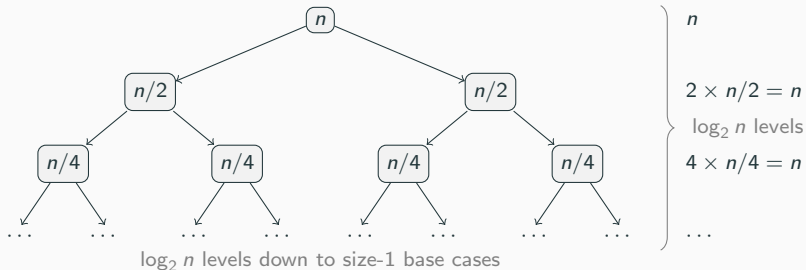

Merge Sort : Code (merge sort)

```
def merge_sort(a):  
    if len(a) <= 1:          # base case  
        return a  
    mid = len(a) // 2  
    left  = merge_sort(a[:mid])  
    right = merge_sort(a[mid:])  
    return merge(left, right)
```

Merging takes n time. Hence, we have the recurrence $T(n) = 2T(n/2) + O(n)$.

Merge Sort : Running Time

Each level of the recursion tree does $O(n)$ total work; there are $\log_2 n$ levels.



Total work = $n \times \log_2 n \Rightarrow$ **running time** = $O(n \log n)$.

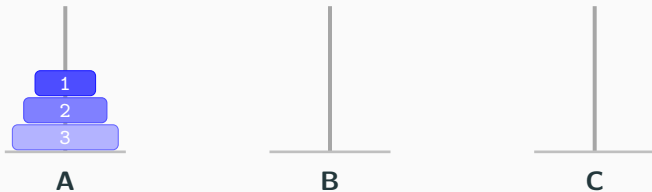
Tower of Hanoi

Tower of Hanoi

Puzzle: move n disks from peg A to peg C using peg B as a helper.

Rules:

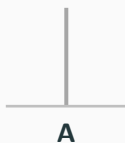
- Move one disk at a time
- Never place a larger disk on top of a smaller one



Tower of Hanoi : Recursive Idea

Base case: $n = 0$: no disks to move, do nothing.

Recursive case: $n \geq 1$: apply the 3-step strategy.



$n = 0$: nothing to do

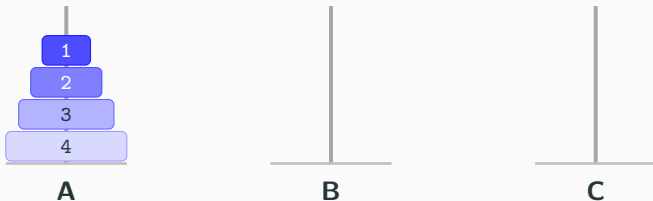


Tower of Hanoi : Recursive Idea

To move n disks from $A \rightarrow C$ using B as helper:

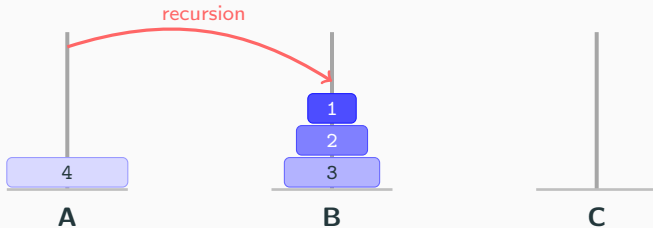
1. Move disks 1 to $n-1$ from $A \rightarrow B$ (recursively)
2. Move disk n (the largest) from $A \rightarrow C$
3. Move disks 1 to $n-1$ from $B \rightarrow C$ (recursively)

Example: $n = 4$, all disks start on A



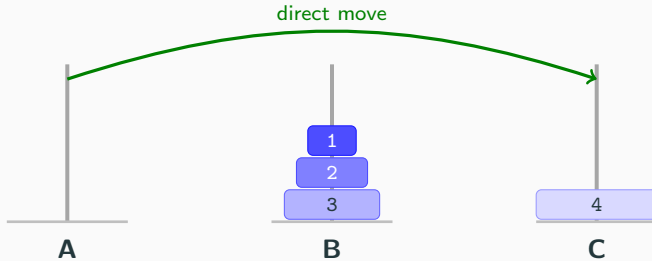
Tower of Hanoi : Recursive Idea

Step 1: move disks 1–3 from A $\rightarrow$ B (recursively)



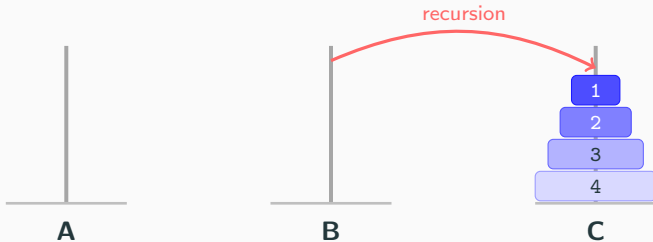
Tower of Hanoi : Recursive Idea

Step 2: move disk 4 from A $\rightarrow$ C (one direct move)



Tower of Hanoi : Recursive Idea

Step 3: move disks 1–3 from B $\rightarrow$ C (recursively) $\Rightarrow$ done!



Tower of Hanoi : Code

```
def hanoi(n, source, target, helper):  
    if n == 0:  
        return  
    hanoi(n - 1, source, helper, target)    # step 1  
    print(f"Move disk {n}: {source} -> {target}")    # step 2  
    hanoi(n - 1, helper, target, source)    # step 3  
  
hanoi(3, "A", "C", "B")  
  
# Move disk 1: A -> C           Move disk 1: B -> A  
# Move disk 2: A -> B           Move disk 2: B -> C  
# Move disk 1: C -> B           Move disk 1: A -> C  
# Move disk 3: A -> C
```

Binary Search

Binary Search

Prerequisite: the array must be **sorted**.

- Compare the target to the **middle** element
- If equal : found
- If target is smaller : search the **left** half
- If target is larger : search the **right** half
- Repeat, halving the search space each time

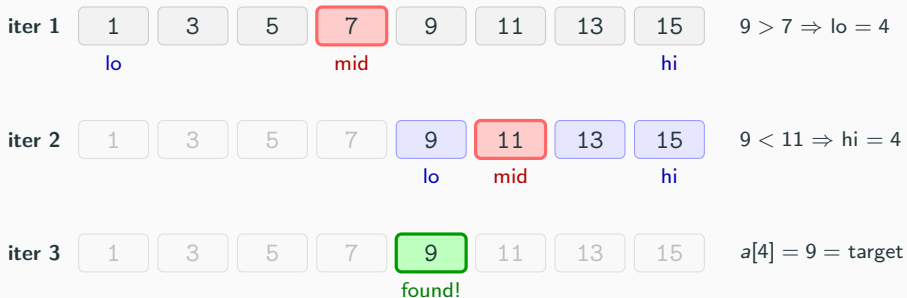


Binary Search : Code

```
def binary_search(a, target):  
    lo, hi = 0, len(a) - 1  
    while lo <= hi:  
        mid = (lo + hi) // 2  
        if a[mid] == target:  
            return mid                # found: return index  
        elif a[mid] < target:  
            lo = mid + 1              # search right half  
        else:  
            hi = mid - 1              # search left half  
    return -1                        # not found
```

- Returns the **index** of target, or -1 if absent
- Each iteration eliminates **half** the remaining elements

Binary Search : Trace (target = 9)



Binary Search : Running Time

Each step halves the search space:

$$n \rightarrow \frac{n}{2} \rightarrow \frac{n}{4} \rightarrow \dots \rightarrow 1$$

After k steps the search space has size $n/2^k$. We stop when $n/2^k = 1$, i.e. $k = \log_2 n$.

Running time = $O(\log n)$.

n	steps
1 000	10
1 000 000	20
10^9	30

Dynamic Programming

Dynamic Programming : Fibonacci

The Fibonacci sequence is defined by:

$$F(0) = 0, \quad F(1) = 1, \quad F(n) = F(n-1) + F(n-2) \text{ for } n \geq 2$$

First values:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...

Goal: given n , compute $F(n)$ efficiently.

Dynamic Programming : Fibonacci

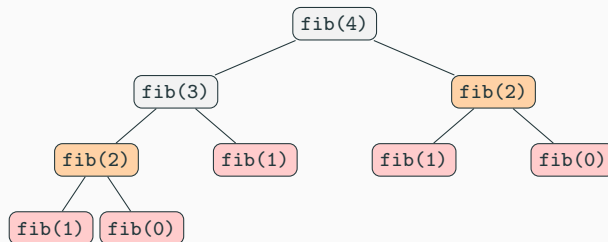
Direct translation of the definition into code:

```
def fib(n):  
    if n <= 1:  
        return n                # base cases:  $F(0)=0$ ,  $F(1)=1$   
    return fib(n-1) + fib(n-2)
```

This is correct, but `fib(n-1)` and `fib(n-2)` each recurse into overlapping subproblems: the same values get recomputed many times.

Dynamic Programming : Fibonacci

Call tree for `fib(4)`: colored nodes are recomputed:



- `fib(2)` computed $2\times$; `fib(1)` computed $3\times$; `fib(0)` computed $2\times$
- In general: $T(n) = T(n-1) + T(n-2) + O(1) \Rightarrow T(n) = O(2^n)$

Dynamic Programming : Fibonacci (DP)

Bottom-up DP: $O(n)$

```
def fib_dp(n):  
    dp = [0] * (n + 1)  
    dp[1] = 1  
    for i in range(2, n + 1):  
        dp[i] = dp[i-1] + dp[i-2]  
    return dp[n]
```

- Fill the table from small to large
- Each entry computed exactly once
- No recursion, no recomputation

0/1 Knapsack : Problem

Given n items, each with a weight w_i and value v_i , and a knapsack of capacity W :

Maximize total value **s.t.** total weight $\leq W$; each item used **at most once**.

Example ($W = 6$):

Item	Weight	Value
1	1	2
2	3	4
3	2	3
4	4	5

Which subset of items maximizes total value without exceeding $W = 6$?

0/1 Knapsack : Problem

Given n items, each with a weight w_i and value v_i , and a knapsack of capacity W :

Maximize total value **s.t.** total weight $\leq W$; each item used **at most once**.

Example ($W = 6$):

Item	Weight	Value
1	1	2
2	3	4
3	2	3
4	4	5

Optimal: items 1, 2, 3 (weight $1 + 3 + 2 = 6$, value $2 + 4 + 3 = \mathbf{9}$).

0/1 Knapsack : Brute Force

Try all $2^4 = 16$ subsets; keep those with total weight ≤ 6 :

Checking all subsets is $O(2^n)$: impractical for large n .

0/1 Knapsack : DP Formulation (1/3)

Subproblem: $dp[i][w]$ = max value using the first i items with weight limit w .

Case 1: skip item i :

$$value = dp[i-1][w]$$

We simply inherit the best value from the previous $i - 1$ items, leaving the weight budget unchanged.

0/1 Knapsack : DP Formulation (2/3)

Case 2: take item i (only if $w \geq w_i$):

$$value = v_i + dp[i-1][w - w_i]$$

We gain value v_i and spend weight w_i , then look up the best value achievable with the remaining capacity $w - w_i$.

0/1 Knapsack : DP Formulation (3/3)

Combined recurrence: take the better of the two cases.

$$dp[i][w] = \max \left(\underbrace{dp[i-1][w]}_{\text{skip}}, \underbrace{v_i + dp[i-1][w-w_i]}_{\text{take, } w \geq w_i} \right)$$

Base case: $dp[0][w] = 0$ for all w (no items available $\Rightarrow$ value is 0).

0/1 Knapsack : Code

```
def knapsack(weights, values, W):  
    n = len(weights)  
    dp = [[0] * (W + 1) for _ in range(n + 1)]  
    for i in range(1, n + 1):  
        for w in range(W + 1):  
            dp[i][w] = dp[i-1][w]           # skip item i  
            if w >= weights[i-1]:  
                take = values[i-1] + dp[i-1][w - weights[i-1]]  
                dp[i][w] = max(dp[i][w], take)  
    return dp[n][W]
```

Time: $O(nW)$ Space: $O(nW)$

Problems

Problem 1 : Lists

Given `scores = [88, 72, 95, 61, 84, 90]`, write code to:

1. Access the third score.
2. Slice the last three scores.
3. Append 77, then sort in descending order.
4. Remove all scores below 80 using a list comprehension.

Problem 2 : Sieve of Eratosthenes

Find all primes up to n by repeatedly crossing out multiples:

1. Start with `is_prime = [True] * (n + 1)`; set `is_prime[0] = is_prime[1] = False`.
2. For each p from 2 to $\sqrt{n}$: if `is_prime[p]`, mark `is_prime[p*p]`, `is_prime[p*p + p]`, ... as False.
3. Collect all indices where `is_prime[i]` is True.

Tasks:

1. Implement `sieve(n)` and verify it returns `[2,3,5,7,11,13]` for $n = 13$.
2. Use it to find the **1000th prime**.
3. Count how many primes are below 10^6 . What is the running time of the sieve?

Problem 3: Coin Change

Given coin denominations, find the **minimum number of coins** to make a target amount (each coin usable any number of times).

Example: coins = [1, 5, 6, 9], amount = 11

Greedy (largest first) gives $9 + 1 + 1 = 3$ coins. Can you do better?

Hint: think about subproblems. If you know the min coins for every amount $< a$, how can you compute the min coins for amount a ?

Tasks:

1. Define the subproblem. Write the recurrence.
2. Implement `coin_change(coins, amount)`.
3. What is the running time?

Problem 4 : Counting Inversions

An **inversion** in a list is a pair (i, j) with $i < j$ but $\text{arr}[i] > \text{arr}[j]$.

```
arr = [3, 1, 2, 5, 4]
```

```
# inversions: (3,1), (3,2), (5,4) -> count = 3
```

1. Write a brute-force solution using two nested loops. What is its time complexity?
2. (*Bonus*) Write an $O(n \log n)$ solution.

Hint: think about merge sort. When merging two sorted halves, every element in the right half that is placed before an element in the left half contributes exactly (number of remaining elements in the left half) inversions.